# NLP Assignment: Data Cleaning, Tokenization, and N-gram Modeling

Report

February 11, 2026

## 1 Task 1

### 1.1 Task 1.1: Data Cleaning

I first looked up common methods to clean large datasets for NLP tasks. I found a standard procedure called **NFC normalization** done on these datasets. I implemented this along with suggestions given in the assignment doc.

The general motivation is that we want the text to be as close to normal language as possible without having random characters that can confuse the model.

**Various cleaning done:**

1. **Unicode normalization:** Unicode has various ways to write the same text. There are some characters that can be represented in multiple ways. We obviously want to avoid this because it can lead to incorrect counts for n-grams, and generally incorrect representations for various tokens. So, what NFC normalization does is convert these different equivalent representations into a single one.

   ```
   line = unicodedata.normalize("NFC", line)
   ```

2. **Removing common invisible characters:** There are various common invisible characters that are often found in text. The full list can be found in the code. These characters are simply removed from the text so that they don't become separate bogus tokens.

   ```
   line = line.replace(ch, "")
   ```

3. **Removing control/format characters:** These are characters with unicode format CC and CF. Control characters could be stuff like NUL characters, ESC character and format characters is stuff like word joiner etc. We remove these.

   ```
   s = "".join(ch for ch in s if unicodedata.category(ch) not in {"Cc", "Cf"})
   ```

4. **Normalizing whitespaces:** (i.e., if there are multiple whitespaces or a tab character etc., then we will just convert all that to a single space.)

   ```
   s = re.sub(r"\s+", " ", s).strip()
   ```

### 1.2 Task 1.2: Tokenization Methods

#### 1.2.1 Whitespace

The whitespace tokenizer is pretty simple. We look through the line and identify if the character is a "word character", which means it checks if it belongs to Letters, Marks or Numbers unicode categories. If no, it is treated as a separate token (unless it is a space obviously). We split if the current character is a space, but don't include it as a token.

### 1.2.2 Regex

The Regex tokenizer is the whitespace tokenizer with some optimisations thrown in.

- **URL Pattern:** `(?:https?://|www)\S+`
  We match anything starting with http, https, or www followed by any non-whitespace characters. Here, we want the entire link to be one thought, not broken up, so it works.
- **Email Pattern:** `[A-Za-z0-9._%+]+@[A-Za-z0-9.]+\.[A-Za-z]{2,}`
  Groups the username, the @ symbol and the domain (gmail etc) into one thing. We want the email address as one token.
- **Numbers:** `(?:\d{1,3}(?:\,\d{3})+(?:\.\d+)?|\d+(?:\.\d+)?)`
  We do this so that comma separated numbers stay together.

### 1.2.3 BPE (Byte Pair Encoding)

We begin by replacing space by the end of word (EOW) token `/w`.

1. We first keep track of all the words in the entire corpus. This number comes up to around 700,000 for both languages. We make a counter called `word_freq` to count how many times a word occurs in the corpus. Then we convert all the words to list form (with each letter being an entry in the list) ending with EOW.
2. The **naive BPE** would proceed as follows: We would first calculate pair wise counts for each pair. Then we would choose the one with the largest count and merge. Then we again calculate pair wise counts for each pair and so on. This however would have a time complexity of $O$(number of merge operations $\times$ number of words) which would be $50,000 \times 700,000$, which is FAR TOO MUCH.
3. **Optimization:** We only calculate pair wise counts for each pair once. After we choose the one with the largest count and merge, we only change the words which have the pair we are merging. This optimization improves speed to where I can run it and it takes a reasonable time.
4. Further optimizations are possible, but I feel the tradeoff of complexity needed to time saved is not worth it.
5. For **encoding**, the naive way is doing the merges in the order we performed them while training. But then for this, again for each token, we will have 10k merge operations, which is not computationally feasible.
6. Instead, for each word, we first split it into a list (e.g., "List" becomes `['L','i','s','t']`). Then we consider what merges we can do and perform the one with highest priority. We keep doing this until we can't anymore. Also, if a word is already used, just cache it and directly used the cached word.
7. **Implementation details:**
   - `word_freq` is a counter that keeps track of words and their frequencies. We use this to create `word_symbols` and `word_freqs`.
   - We use `pair_counts` to calculate frequency of all the pair counts. We also use `pair_to_words` to keep track of which word (specifically `w_id`) contains the pair.
   - In the merge loop, we get the best pair while ensuring we aren't merging EOW or punctuation. We temporarily subtract contributions, apply the merge, and re-add contributions to ensure correct counting.

### 1.3 Task 1.3: Qualitative Analysis

#### 1.3.1 Whitespace

- **Sensible:** In English, words are separated by spaces. So separating by spaces makes semantic sense. For example: "I am Ronit", "The sun is bright", "  " are correctly split up into semantic units.
- **Insensible:** The most common failing is that it messes up with punctuation. "Ronit" and "Ronit." are treated differently. A severe example would be "done" and "done." becoming two separate tokens. It also keeps words like "don't" as one token.

#### 1.3.2 Regex Tokenization

- **Sensible:** Fixes the "don't" problem. Good for separating URLs properly (e.g., "https://www.google.com Also ensures "done." and "done" are not separate tokens.
- **Insensible:** Specific words like "U.S.A" would be split into U S A. Mongolian has English and Russian loanwords appended with dashes (e.g., "e-"), which regex might split despite having one semantic meaning.

#### 1.3.3 BPE Tokenization

- **Sensible:** Better captures morphology where common prefixes/suffixes like "un", "ing" become their own tokens. It compresses vocabulary. In Mongolian, it helps separate the root (e.g., "") from grammatical markers (e.g., "").
- **Insensible:** BPE uses frequency as a heuristic, not semantic meaning. It might tokenize "thematic" as "the + matic". It struggles with casual spellings (e.g., "AAAAAAAHH-HHH") and could treat common words as single tokens, reducing predictive power (e.g., "strawberry").

## 2 Task 2

Other than the one mentioned in the problem statement PDF, there is another assumption we implicitly make when using 4-grams: we consider that the previous 4 tokens contain all the relevant information for the current token. Mathematically we make the **Markov assumption**:

$$P(t_n \mid t_1, t_2, \ldots, t_{n-1}) = P(t_n \mid t_{n-1}, t_{n-2}, t_{n-3})$$

### 2.1 Task 2.1: Training and Prediction

1. **Training:** We calculate how many times a particular tuple of length 1-4 has appeared using dictionaries (`unigram`, `trigram`, etc.). We also track continuations (`prev3`, `prev2`).
2. **Prediction:** This is an autocomplete problem. We feed initial tokens and choose the most probable next token by looking at the past 3 tokens. If unseen, we "backoff" to 2 tokens, and so on.

### 2.2 Task 2.2: Smoothing Techniques

#### 2.2.1 No Smoothing with Backoff

We calculate the probability of the correct target word. If there are no instances in the training set even for unigrams, we use a floor probability of $1e-18$ to avoid infinity while observing high perplexity.

### 2.2.2 Kneser-Ney Smoothing

The intuition (e.g., "Francisco") is that some high-frequency words only appear in specific contexts. We balance raw frequency by considering the "fertility" of a token. **Formula:**

$$P(w \mid h) = \frac{\max(c(w \mid h) - D, 0)}{c(h)} + \beta \cdot P(w \mid h_{i-1})$$

where $\beta = \frac{\text{discount} \cdot t_h}{c(h)}$ and $t_h$ is the number of unique continuation tokens.

### 2.2.3 Witten-Bell Smoothing

Sequences with many different following tokens are more diverse. Witten-Bell gives probability mass to unseen words proportional to the number of observed types ($T$). **Formula:**

$$P(w \mid \text{context}) = \lambda \cdot P_{ML} + (1 - \lambda) \cdot P_{\text{continuation}}$$

where $\lambda = \frac{N}{N+T}$.

## 2.3 Task 2.3: Perplexity

Perplexity is the "branching factor," defined as:

$$\text{PP}(W) = \left( \prod_{i=1}^{n} P(w_i \mid w_{i-1}, \dots) \right)^{-\frac{1}{n}} = \exp\left( -\frac{1}{n} \sum \log P(w_i \mid \dots) \right)$$

## 2.4 Task 2.4: Results and Analysis

Table 1: Perplexity Results with $\epsilon = 10^{-100}$

| **Smoothing Method** | **BPE** | **WS** | **Regex** |
|---|---|---|---|
| None | 15,088,052,286.34 | 8.77e15 | 4.53e16 |
| Kneser-Ney | 280.07 | 2,071.69 | 2,682.35 |
| Witten-Bell | 305.94 | 2,304.56 | 3,007.43 |

Table 2: Perplexity Results with $\epsilon = 10^{-12}$

| **Smoothing Method** | **BPE** | **WS** | **Regex** |
|---|---|---|---|
| None | 395.46 | 1,755.54 | 2,230.58 |
| Kneser-Ney | 280.07 | 530.34 | 611.93 |
| Witten-Bell | 305.94 | 589.95 | 686.09 |

**Analysis:**

- **BPE is the best:** It dominates because it eliminates OOV (Out-of-Vocabulary) errors by breaking rare words into common sub-units.
- **Kneser-Ney vs. Witten-Bell:** KN wins because it accounts for "fertility." WB is a decent runner-up but more reactive to raw counts.
- **The "None" Smoothing Crash:** Without smoothing, the model relies entirely on $\epsilon$. At $10^{-100}$, the penalty is so massive the perplexity hits astronomical numbers.
- **Regex & WS Sparsity:** These struggle because variations of the same word (e.g., "done" vs "done.") are treated as separate tokens, diluting counts and increasing confusion.